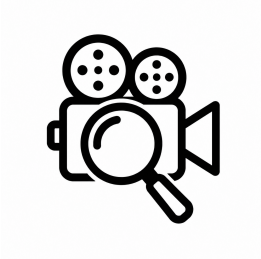
Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

IC-8060 Recuperación de Información Textual



Progra01: Creación del índice de búsqueda

Encuadre

Profesor:

Aurelio Sanabria

Estudiantes:

Santiago Espinoza Rendon	2024156530
Fabián Araya Ortega	2019159110
Mauricio González Prendas	2024143009

Fecha:

Noviembre de 2026

II Semestre

2026

Índice

1. Políticas de normalización **1**
1.1. Tokenización y limpieza previa 1
1.2. Conversión a minúsculas 1
1.3. Filtrado de palabras vacías 1
1.4. Reducción morfológica multilingüe 1
1.5. Normalización de entidades temporales y números 2
1.6. Tesauro de equivalencias audiovisuales 2

2. Estadísticas y evaluación de rendimiento de los índices **2**
2.1. Métricas empíricas de los tres índices 3
2.2. Análisis comparativo de resultados 3

3. Descripción de la estructura interna del archivo del índice **4**
3.1. Fundamentación del formato binario manual 4
3.2. Encabezado global de 64 bytes 4
3.3. Tabla de documentos y normas vectoriales 4
3.4. Diccionario o vocabulario 6
3.5. Listas de ocurrencias o postings 7
3.6. Inspección automatizada de la estructura binaria 8

1. Políticas de normalización

Con el propósito de transformar las cadenas de texto heterogéneas recolectadas por la araña en unidades léxicas homogéneas y comparables, el indexador articula sus decisiones de preprocesamiento mediante un componente modular de normalización, el cual permite habilitar o prescindir de distintos filtros según los parámetros definidos en la configuración.

1.1. Tokenización y limpieza previa

Previo a la segmentación léxica, el texto atraviesa una etapa de saneamiento basada en el estándar Unicode NFKC, orientada a descomponer caracteres de ancho completo y ligaduras tipográficas como *fi* o *fl*, homologando a su vez comillas curvas y guiones tipográficos. A partir del texto limpio, la división en tokens se lleva a cabo mediante una expresión regular que reconoce marcadores semánticos entre corchetes angulares y admite palabras con apóstrofes o guiones internos, elementos habituales dentro del dominio audiovisual en términos como *sci-fi*, *post-credits* o *spider-man*:

```
re_token = r"<[A-Z0-9_]+>|[^\W_]+(?:[-'_][^\W_]+)*"
```

1.2. Conversión a minúsculas

A fin de evitar dispersión léxica y asegurar que variantes como *“Cinema”*, *“CINEMA”* y *“cinema”* converjan hacia una misma entrada del diccionario, los tokens alfanuméricos se transforman a minúsculas de manera uniforme, preservando intactos únicamente los marcadores sintéticos generados por el propio motor.

1.3. Filtrado de palabras vacías

Con el fin de remover términos de altísima frecuencia pero nulo valor discriminatorio, se estructuró un catálogo exhaustivo de palabras vacías para español e inglés que abarca artículos, preposiciones, conjunciones, pronombres y verbos auxiliares, resolviendo la pertenencia de cada término en tiempo constante $O(1)$ mediante conjuntos inmutables en memoria.

1.4. Reducción morfológica multilingüe

Para colapsar las variantes flexivas y derivativas hacia una raíz común, se incorporó el algoritmo Snowball para inglés y español, apoyado en un mecanismo heurístico que determina el idioma predominante a partir de la proporción de palabras vacías y la presencia de grafías propias del español como tildes o la letra eñe.

Respecto a la eficiencia computacional del proceso, durante las primeras pruebas sobre guiones que superaban los noventa mil caracteres, la ejecución reiterada del algoritmo sobre palabras sumamente repetidas representaba el mayor costo computacional del proceso; por ello, se implementó una tabla de dispersión en memoria que almacena las raíces ya calculadas, evitando procesar nuevamente términos conocidos y reduciendo el tiempo de normalización en más de un 80 %.

1.5. Normalización de entidades temporales y números

Dado que en las búsquedas audiovisuales las referencias cronológicas suelen ser determinantes para acotar estrenos o épocas de producción, el normalizador incorpora reglas para distinguir fechas completas y años de interés histórico —acotados entre 1888 y 2099—, sustituyéndolos respectivamente por etiquetas sintéticas como `<FECHA>` o `<ANIO_YYYY>`, mientras que las cantidades numéricas generales se homologan bajo el identificador `<NUM>` o se omiten en función de la política activa.

1.6. Tesauro de equivalencias audiovisuales

Con el propósito de tender un puente entre consultas breves y documentos redactados en distintos estilos o idiomas, se elaboró un tesauro enfocado en la producción cinematográfica y televisiva que canaliza sinónimos habituales hacia un único lema canónico, permitiendo que variantes terminológicas confluyan al mismo concepto:

- *película, filme, motion picture, movie, cinema* ⟶ `film`
- *director, cineasta, filmmaker, realizadora* ⟶ `realizador`
- *screenplay, guion, libreto, script* ⟶ `screenplay`
- *actor, actriz, reparto, star, intérprete* ⟶ `interprete`
- *soundtrack, ost, banda sonora, score* ⟶ `banda_sonora`

2. Estadísticas y evaluación de rendimiento de los índices

Con el propósito de evaluar experimentalmente el impacto de las decisiones lingüísticas sobre el índice, se estructuraron tres perfiles en archivos de configuración independientes:

- **Política 1, mínima:** concebida como línea base, se limita a la tokenización alfanumérica y al uso exclusivo de minúsculas, prescindiendo deliberadamente del filtrado de palabras vacías, la lematización y la detección de entidades.
- **Política 2, estándar:** orientada al modelo clásico de recuperación de información, combina minúsculas, supresión de puntuación, filtrado de palabras vacías en ambos idiomas y reducción morfológica mediante Snowball.
- **Política 3, avanzada:** diseñada para enriquecer la semántica del dominio audiovisual, añade a los filtros anteriores la normalización de fechas, la detección de años históricos y la unificación de sinónimos a través del tesauro.

La construcción de cualquiera de los índices a partir de su archivo de configuración se ejecuta de forma desacoplada mediante la herramienta de línea de comandos:

```
uv run indexador indexar --config configuraciones/estandar.conf --limite 1000
```

Listing 1: Construcción del índice mediante archivo de configuración.

2.1. Métricas empíricas de los tres índices

Con el fin de contrastar las tres políticas bajo condiciones controladas sobre una muestra de quinientos documentos, se utilizó el procedimiento de evaluación automatizado:

```
uv run indexador benchmark --limite 500
```

Listing 2: Ejecución comparativa de las tres políticas de indexación.

Los indicadores cuantitativos resultantes de la ejecución se resumen en la Tabla 1.

Tabla 1: Comparativa cuantitativa de los tres índices generados.

Política	Docs	Vocabulario	Postings	Tamaño	Tiempo	RAM	Bytes/Doc
`minimo`	500	88 018	600 454	9,40 MB	6,49 s	86,9 MB	19 708,6
`estandar`	500	59 224	459 396	6,91 MB	10,48 s	87,6 MB	14 496,2
`avanzado`	500	57 619	451 469	6,78 MB	15,96 s	88,4 MB	14 222,7

2.2. Análisis comparativo de resultados

Al examinar las métricas registradas, se observan contrastes claros originados por el grado de normalización aplicado:

1. **Compresión léxica y economía de almacenamiento:** la combinación de palabras vacías y reducción morfológica en la política estándar contrajo el vocabulario en un 32.7 % frente a la línea base, pasando de 88 018 a 59 224 términos únicos, lo cual derivó en un ahorro del 26.5 % en el espacio en disco al disminuir el archivo de 9,40 MB a 6,91 MB y suprimir más de ciento cuarenta y un mil ocurrencias redundantes.

2. **Consolidación semántica del tesauro:** en la política avanzada, el vocabulario experimentó una compactación complementaria hasta situarse en 57 619 términos, fruto de encauzar términos afines y variantes lingüísticas hacia lemas comunes, incrementando la densidad semántica de cada lista de ocurrencias.

3. **Estabilidad en el uso de memoria:** a pesar de procesar documentos voluminosos con decenas de miles de palabras cada uno, la memoria residente requerida por el indexador se mantuvo estable alrededor de los 87 a 88 MB, corroborando que el esquema de partición en bloques temporales y su posterior integración externa evitan la saturación de los recursos del sistema.

Para comprobar la operatividad del modelo vectorial y la validez de las normas euclidianas calculadas, es posible formular consultas puntuales resueltas mediante similitud coseno directamente sobre el archivo binario generado:

```
uv run indexador buscar salida/indice_estandar.idx "film director" --top 5
```

Listing 3: Consulta vectorial con similitud coseno sobre el índice.

3. Descripción de la estructura interna del archivo del índice

3.1. Fundamentación del formato binario manual

En correspondencia con las directrices de la especificación técnica, el índice desarrollado prescinde de bases de datos embebidas o módulos de serialización automática, administrándose como un archivo binario monolítico con extensión `.idx` que se escribe y lee directamente a nivel de bytes mediante empaquetamiento Little-Endian con el módulo `struct`.

La arquitectura del formato se fundamenta en dos principios rectores:

1. **Alineación deliberada para inspección con `hexdump -C`:** el encabezado se reservó en sesenta y cuatro bytes exactos para cubrir con precisión las primeras cuatro líneas de dieciséis bytes del visor hexadecimal, mientras que los registros documentales y de ocurrencias poseen dimensiones fijas de cuarenta y cuatro y doce bytes respectivamente, almacenando las cadenas del vocabulario en codificación UTF-8 para su lectura directa en la columna ASCII.

2. **Capacidad de acceso aleatorio:** la uniformidad en las estructuras permite consultar la norma vectorial o los metadatos de cualquier documento en tiempo constante calculando su posición física como offset $= 64 + (\text{doc_id} \times 44)$, al tiempo que la ordenación alfabética del léxico posibilita búsquedas binarias en $O(\log |V|)$ y saltos directos hacia los bloques de ocurrencias asociados.

3.2. Encabezado global de 64 bytes

Los primeros sesenta y cuatro bytes del archivo corresponden a los metadatos globales del índice, situados en el intervalo comprendido entre `0x00000000` y `0x0000003F`, accesibles mediante el comando `hexdump -C -n 64 salida/indice_estandar.idx`:

```
00000000  45 4e 43 55 41 44 52 45  01 00 0f 00 f4 01 00 00  |ENCUADRE........|
00000010  58 e7 00 00 84 02 07 00  00 00 00 00 40 00 00 00  |X...........@...|
00000020  00 00 00 00 30 56 00 00  00 00 00 00 b3 7a 1a 00  |....0V.......z..|
00000030  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
```

Listing 4: Volcado del encabezado mediante `hexdump`.

3.3. Tabla de documentos y normas vectoriales

A partir de la posición `0x00000040` se despliega la tabla continua de documentos y normas vectoriales, conformada por entradas contiguas de cuarenta y cuatro bytes empaquetadas bajo el formato `<I32sIf`:

- `doc_id`: entero sin signo de 32 bits y 4 bytes que representa el identificador secuencial continuo.
- `hash_url`: arreglo de 32 bytes con la clave SHA-256 binaria del documento.

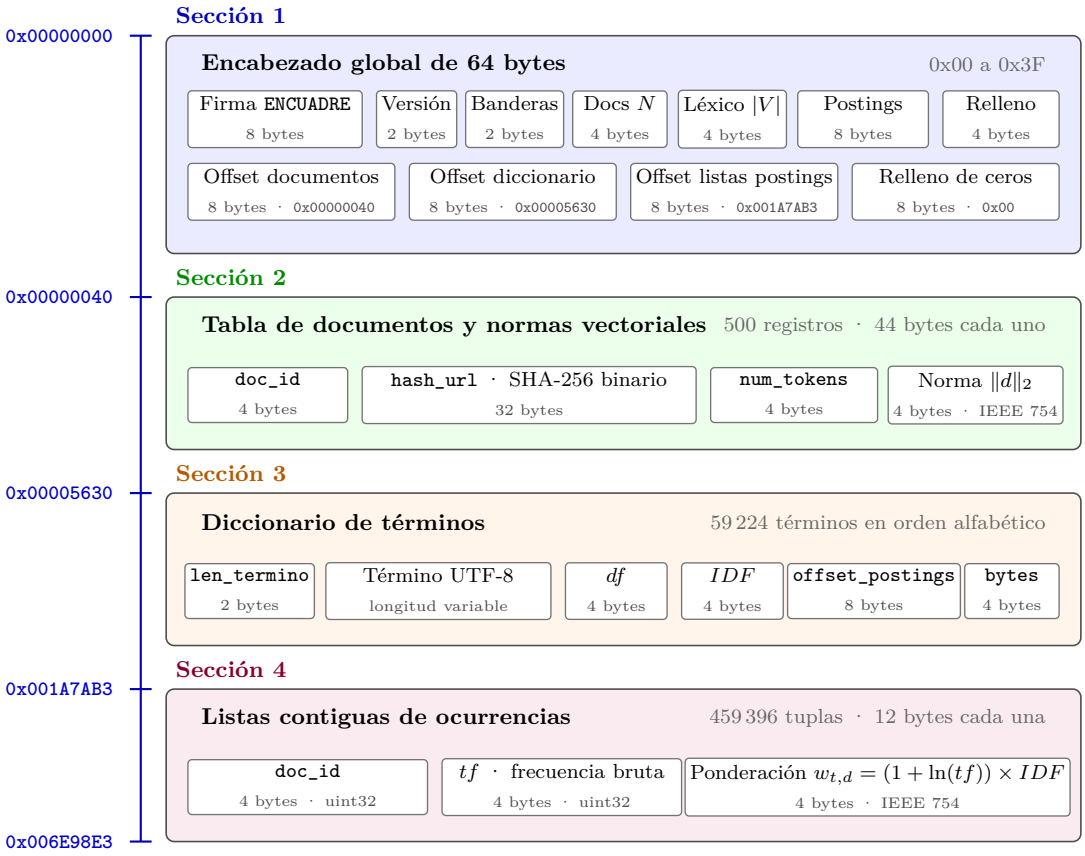


Figura 1: Disposición secuencial de las cuatro secciones en el archivo binario `.idx`.

Tabla 2: Mapeo campo por campo del encabezado del archivo `.idx`.

Desplazamiento	Tipo	Bytes	Interpretación y valor decodificado
`0x00 - 0x07`	`char[8]`	8	Firma mágica de identificación: `ENCUADRE`
`0x08 - 0x09`	`uint16`	2	Versión del esquema binario: 1
`0x0A - 0x0B`	`uint16`	2	Banderas de normalización: 0x000F con minúsculas, puntuación, palabras vacías y lematización activadas
`0x0C - 0x0F`	`uint32`	4	Total de documentos N: 500
`0x10 - 0x13`	`uint32`	4	Tamaño del vocabulario $\|V\|$: 59 224 términos
`0x14 - 0x1B`	`uint64`	8	Total de ocurrencias acumuladas: 459 396 postings
`0x1C - 0x23`	`uint64`	8	Desplazamiento a la tabla de documentos: byte 64, equivalente a 0x40
`0x24 - 0x2B`	`uint64`	8	Desplazamiento a la sección del diccionario: byte 22 064, equivalente a 0x5630
`0x2C - 0x33`	`uint64`	8	Desplazamiento a la sección de postings: byte 1 735 347, equivalente a 0x1A7AB3
`0x34 - 0x3F`	`byte[12]`	12	Relleno de ceros para alineación estricta a 64 bytes

- `num_tokens`: entero sin signo de 32 bits y 4 bytes con el conteo total de tokens del documento.
- `norma_vectorial`: flotante de 4 bytes con la norma euclidiana $\|d\|_2 = \sqrt{\sum w_{t,d}^2}$.

```
00000040  00 00 00 00 df 3c 0a bb  ab e9 ad 1e 5a a2 0b eb  |.....<......Z...|
00000050  3e f1 5f 63 59 07 48 16  c4 71 75 14 6d fa a8 20  |>._cY.H..qu.m.. |
00000060  ac ce 46 b6 06 35 00 00  b8 0b 9f 43 01 00 00 00  |..F..5.....C....|
00000070  08 67 35 fc f8 5c 56 b5  f3 fd 10 2c 83 71 19 7a  |.g5..\V....,.q.z|
00000080  bf a2 11 ec 21 90 9d ff  5a b5 99 da 83 37 0c c2  |....!...Z....7..|
00000090  a4 0d 00 00 31 b4 34 43                          |....1.4C|
```

Listing 5: Volcado de los dos primeros registros documentales mediante `hexdump`.

Al examinar detalladamente los primeros dos registros del volcado anterior, se aprecian sus componentes correspondientes:

- **Documento 0 en el rango `0x00000040` a `0x0000006B`:** identificador secuencial cero, resumen criptográfico SHA-256 almacenado en treinta y dos bytes, longitud calculada de 13 574 tokens y una norma vectorial $\|d\|_2 = 318{,}0916$ representada por la secuencia en coma flotante `b8 0b 9f 43`.
- **Documento 1 en el rango `0x0000006C` a `0x00000097`:** identificador secuencial uno, resumen SHA-256, longitud de 3 492 tokens y norma $\|d\|_2 = 180{,}7039$ codificada en los bytes `31 b4 34 43`.

3.4. Diccionario o vocabulario

En la dirección `0x00005630`, obtenida al sumar el bloque de cabecera y las quinientas entradas documentales previas, comienza la sección del léxico, donde cada término se estructura con una longitud dinámica según la extensión de su cadena:

- `len_termino`: entero de 16 bits y 2 bytes con la longitud de la cadena en UTF-8.
- `termino_utf8`: bytes legibles directamente en la columna ASCII del visor hexadecimal.
- `df`: entero de 32 bits y 4 bytes con la frecuencia documental.
- `idf`: flotante de 4 bytes con el peso inverso documental calculado según $\ln(N/df + 1)$.
- `offset_postings`: entero de 64 bits y 8 bytes con el desplazamiento absoluto al bloque de ocurrencias.
- `bytes_postings`: entero de 32 bits y 4 bytes con la longitud en bytes de la lista.

```
00005630  01 00 30 01 00 00 00 70  ee c6 40 b3 7a 1a 00 00  |..0....p..@.z...|
00005640  00 00 00 0c 00 00 00 03  00 30 2d 30 01 00 00 00  |.........0-0....|
00005650  70 ee c6 40 bf 7a 1a 00  00 00 00 00 0c 00 00 00  |p..@.z..........|
00005660  0d 00 30 2d 31 32 2d 34  39 30 36 33 30 2d 33 01  |..0-12-490630-3.|
00005670  00 00 00 70 ee c6 40 cb  7a 1a 00 00 00 00 00 0c  |...p..@.z.......|
```

Listing 6: Volcado del diccionario de términos mediante `hexdump`.

Al contrastar las entradas consecutivas capturadas en el volcado anterior, se comprueba la serialización individual de cada término:

- **Término `"0"`:** longitud de 1 byte con valor `01 00`, carácter ASCII `'0'` con código `30`, $df = 1$, peso inverso $IDF = \ln(500/1 + 1) = 6{,}2166$ en los bytes `70 ee c6 40`, desplazamiento de postings en `0x001A7AB3` y tamaño de postings de 12 bytes.
- **Término `"0-0"`:** longitud de 3 bytes con valor `03 00`, cadena ASCII legible `"0-0"`, desplazamiento de ocurrencias en `0x001A7ABF` y tamaño de 12 bytes.
- **Término `"0-12-490630-3"`:** longitud de 13 bytes con valor `0d 00`, caracteres visibles en el visor ASCII `"0-12-490630-3"` y desplazamiento de ocurrencias en `0x001A7ACB`.

3.5. Listas de ocurrencias o postings

A partir del byte `0x001A7AB3` se despliegan las listas de ocurrencias agrupadas por término, en las cuales cada posting individual se compacta en doce bytes según el patrón `<IIf`:

- `doc_id`: identificador de documento de 32 bits y 4 bytes.
- `tf`: frecuencia bruta del término en el documento, en 4 bytes.
- `peso_tfidf`: peso ponderado $w_{t,d} = (1 + \ln(tf)) \times IDF$ en 4 bytes.

```
001a7ab3  eb 01 00 00 01 00 00 00  70 ee c6 40 29 01 00 00  |........p..@)...|
001a7ac3  02 00 00 00 fa 68 28 41  93 00 00 00 01 00 00 00  |.....h(A........|
001a7ad3  70 ee c6 40 52 00 00 00  04 00 00 00 bc 5a 6d 41  |p..@R........ZmA|
```

Listing 7: Volcado de la lista de ocurrencias mediante `hexdump`.

Tabla 3: Mapeo de registros de postings consecutivos.

Offset hex	**Doc**	**TF**	**Hex IEEE 754**	**Peso $w_{t,d}$**	**Término**
`0x001A7AB3`	491	1	`70 ee c6 40`	6,2166	`"0"`
`0x001A7ABF`	297	2	`fa 68 28 41`	10,5256	`"0-0"`
`0x001A7ACB`	147	1	`70 ee c6 40`	6,2166	`"0-12-490630-3"`
`0x001A7AD7`	82	4	`bc 5a 6d 41`	14,8346	`"0-2"`

La correspondencia del modelo matemático se comprueba verificando la coincidencia entre la ponderación teórica y su registro binario; por ejemplo, para el término `"0-0"` presente en el documento doscientos noventa y siete con una frecuencia $tf = 2$:

$$w_{t,d} = (1 + \ln(2)) \times 6{,}2166 = (1 + 0{,}693147) \times 6{,}2166 \approx 10{,}5256 \tag{1}$$

cuyo valor en coma flotante Little-Endian IEEE 754 genera con exactitud la secuencia `fa 68 28 41`, equivalente a `0x412868fa` y presente de forma idéntica en el archivo físico.

3.6. Inspección automatizada de la estructura binaria

Complementariamente a la lectura directa de bytes mediante `hexdump`, el indexador provee una herramienta de línea de comandos diseñada para verificar la consistencia del archivo, comprobar los desplazamientos calculados y recuperar los postings de cualquier término en tiempo $O(\log |V|)$ sin cargar el índice completo a memoria:

```
uv run indexador inspeccionar salida/indice_estandar.idx --termino "film"
```

Listing 8: Inspección de estructuras y términos en el archivo binario.